Projet de fin de formation —

Spécialisation Data Scientist

Scoring de défaut de crédit + POC GenAI « Talk to my Data » (Python only)

Version apprenant — Février 2026

Contexte

Vous endossez le rôle de Data Scientist au sein d'une banque de détail. La direction « Recouvrement & Risque » souhaite renforcer sa politique de relance et de recouvrement au prochain trimestre.

Votre mission se déroule en 3 étapes : (1) prise en main du dataset et mise en place d'un dépôt Git reproductible, (2) construction d'un modèle de prédiction du défaut de crédit en tenant compte du déséquilibre de classes, puis (3) réalisation d'un POC GenAI de type « Talk to my Data » sur les mêmes données afin de tester un assistant d'analyse piloté en langage naturel.

Le jeu de données utilisé est un extrait tabulaire (2965 lignes, 26 colonnes) issu d'un dataset public de défaut de carte de crédit, disponible sur BigQuery (ML datasets).

Objectifs pédagogiques

- Structurer un dépôt Git professionnel et reproductible (environnement, exécution, traçabilité).
- Réaliser une analyse exploratoire (EDA) utile pour comprendre les variables et la cible.
- Construire une pipeline ML propre (préparation, split, métriques) et un modèle prêt à être mis en production.
- Traiter explicitement le déséquilibre des classes et justifier une stratégie de seuil / ciblage.
- Mettre en place un POC « Talk to my Data » (non-RAG) avec LangChain v1, capable de répondre à des questions simples sur le dataset, avec traçabilité via le code Python exécuté.

Données et cible

Le dataset contient des informations socio-démographiques (sex, education_level, marital_status, age), des informations financières (limit_balance, bill_amt_1..6, pay_amt_1..6) et des historiques de paiement (pay_0, pay_2..6).

Variable cible : default_payment_next_month (0/1) — défaut le mois suivant.

Une colonne predicted_default_payment_next_month (str) est présente : elle ne doit pas être utilisée comme feature (risque de fuite / leakage) et servira uniquement à illustrer des contrôles de qualité si besoin.

Schéma (rappel) :

> id (float), limit_balance (float), sex (int), education_level (int), marital_status (int), age (float), pay_0 (float), pay_2 (float), pay_3 (float), pay_4 (float), pay_5 (int), pay_6 (int), bill_amt_1..6 (float), pay_amt_1..6 (float), default_payment_next_month (int), predicted_default_payment_next_month (str)

Étape 1 — Création du repo Git + EDA

Objectif : mettre en place un dépôt propre et reproductible, puis comprendre le dataset (qualité, distributions, relations avec la cible). Cette étape se fait exclusivement en Python (notebook).

Travail attendu :

1. Initialiser le dépôt Git (README, .gitignore, structure de dossiers) et réaliser au moins 3 commits propres (setup, EDA, baseline).
2. Charger la table (BigQuery export ou fichier local) et produire un mini dictionnaire de données (définition, type, exemple).
3. Contrôler la qualité : valeurs manquantes, doublons, valeurs aberrantes, cohérences (âges plausibles, montants >= 0, etc.).
4. Explorer la cible : taux de défaut global, par segments (âge, sexe, niveau d'éducation, statut marital, etc.).
5. Définir votre protocole d'évaluation : split train/valid/test (ou CV), métriques retenues, et première hypothèse sur la gestion du déséquilibre.

Étape 2 — Modèle de défaut de crédit (production-ready)

Objectif : livrer un modèle de scoring prêt à être mis en production, en tenant compte du déséquilibre des classes et d'une décision de recouvrement.

Travail attendu :

6. Construire une pipeline de préparation (scikit-learn recommandé) : séparation X/y, split, imputation si nécessaire, scaling/encodage si besoin.
7. Choisir des métriques adaptées au déséquilibre : PR-AUC (prioritaire), recall/precision, matrice de confusion, et une métrique orientée décision (ex. recall@topK).
8. Traiter le déséquilibre : class_weight, sous-échantillonnage/sur-échantillonnage (au choix) et justification.
9. Entraîner un modèle baseline (ex. LogisticRegression) puis un modèle amélioré (ex. arbre/boosting) si pertinent.
10. Choisir un seuil de décision : proposer une stratégie claire (top K% à relancer, ou seuil de probabilité) alignée avec un coût métier.
11. Sauvegarder le modèle et produire un fichier de scoring (id, proba_default, label_pred) sur un jeu de test.

Étape 3 — POC GenAI « Talk to my Data » (LangChain v1 + Streamlit)

Objectif : construire un assistant d'analyse tabulaire (non-RAG) qui répond à des questions simples sur le dataset en langage naturel, en générant et exécutant du code Python (pandas) sur un DataFrame.

Contraintes obligatoires : LangChain v1, pas de SQL, pas d'accès réseau, pas d'écriture disque. Chaque réponse doit afficher le code Python exécuté et un extrait du résultat (table, agrégat, ou graphique).

L'interface doit être réalisée avec Streamlit.

Exemples d'agents simples (architecture minimale)

ProfilingAgent — Répond aux questions de type « taux de défaut global », « distribution de age », « top valeurs de education_level ». Génère du code pandas + affichage (describe, value_counts, hist).

SegmentationAgent — Calcule des taux de défaut par segment (ex. par tranche d'âge, sexe, marital_status) et retourne un tableau trié + un graphique simple.

ComparisonAgent — Compare défaut vs non-défaut sur quelques variables numériques (moyenne/médiane, boxplot) et met en avant les écarts.

ModelInsightAgent — Expose des informations de base sur le modèle entraîné (features utilisées, importance simple, performance sur test) sans refaire de training dans l'app.

Travail attendu :

12. Mettre en place un agent simple LangChain v1 avec un outil unique d'exécution Python contrôlée (DataFrame en mémoire).
13. Imposer un format de sortie : (1) Réponse en français, (2) Code Python exécuté, (3) Résultat (table/graph).
14. Créer un « golden set » d'au moins 10 questions de validation et vérifier que l'assistant produit des résultats corrects (comparaison avec calculs hors agent).
15. Exposer le système dans une app Streamlit (champ question, affichage réponse, code, résultats, historique).
16. Gérer les refus : si la question sort du périmètre du dataset ou demande une info absente, l'assistant doit répondre « impossible avec les données disponibles ».

Livrables et structure du dépôt Git

Livrables attendus :

1) Dépôt Git complet (README, .gitignore, historique de commits lisible).

2) Notebooks : EDA (étape 1), entraînement (étape 2), amélioration (optionnel).

3) Artefacts : modèle sauvegardé (joblib) + script d'inférence.

4) POC GenAI : app Streamlit + code LangChain v1 (agents, prompts, outils).

5) Rapport synthétique : métriques, seuil/topK, limites et recommandations.

Organisation recommandée du dépôt :

```
/
|-- README.md
|-- data/
|   `-- raw/ (export BigQuery ou CSV/Parquet)
|-- notebooks/
|   |-- 01_setup_repo_et_eda.ipynb
|   |-- 02_modelisation_baseline.ipynb
|   `-- 03_modelisation_advanced.ipynb
|-- src/
|   |-- data_prep.py
|   |-- train.py
|   |-- infer.py
|   |-- metrics.py
|   `-- config.py
|-- app/
|   |-- streamlit_app.py
|   `-- agents/ (langchain v1)
|       |-- prompts.py
|       |-- tools.py
|       `-- agent.py
|-- models/
|   `-- model.joblib
|-- reports/
|   |-- figures/
|   `-- model_report.md
`-- requirements.txt
```

Grille d'évaluation (synthèse)

- Cadrage & traçabilité : repo propre, reproductible, commits lisibles, hypothèses explicites.
- EDA : contrôles qualité, analyses pertinentes, visualisations lisibles, compréhension de la cible.
- Pipeline ML : pas de leakage, split/validation rigoureux, métriques adaptées au déséquilibre.
- Modèle & décision : performance, calibration si pertinent, choix de seuil/topK justifié.
- POC GenAI : LangChain v1, python-only, affichage systématique du code, robustesse et refus corrects.

- Qualité générale : lisibilité, factorisation, commentaires utiles, clarté du README.